

Proj-Read-Write-Features Terminology

This a running doc of the terminology convention that I would like to use and some of the mathematical theory behind things.

1. Terminology

- L is the set of layers
- J is the set of attention heads
- d_{model} is residual-stream width.
- d_{head} is per-head width.
- $a_{\ell,A} \in \mathbb{R}^{d_{\text{model}}}$ is the activation at layer ℓ for task A
- h is a head
- $\mathcal{T}$ is the task universe (antonym, synonym, English–French, country–capital, ...) and $A \in \mathcal{T}$ is a task.
- p_A^j is the j -th prompt for task A
- $\mathcal{P}_A = \{p_A^j\}_j$ is the prompt set for A (varying in ICL context length, unless explicitly mentioned otherwise)
- $h(p_A^j) \in \mathbb{R}^{d_{\text{model}}}$ is the head activation at the final token position (final cue token) on prompt p_A^j .
- The head vector $h_A = \frac{1}{|\mathcal{P}_A|} \sum_j h(p_A^j)$ is the average head activation for task A .
- $v_A = \sum_{h \in H} h_A$ is the function vector for task A where H is the selected subset of heads for our definition of function vectors.
- $v_A^j = \sum_{h \in H} h(p_A^j)$ is the per prompt function vector for prompt j on task A .
- z_ℓ^t is the residual stream at layer ℓ at token t .

2. Read Directions

If we treat the span of $\{v_A\}_{A \in \mathcal{T}}$ as the task imitation space, then we want to get the task identification space. This is some causal subspace (ideally occurring at earlier tokens at earlier layers) which feeds into the model machinery to output the task imitation space. To do this, we can decompose each v_A into it's head vectors $\sum_{h \in H} h_A$. It is then possible to compute the *read direction* (previously called *d_payload*) for each h_A . The idea is that the read direction, r_A^h , is the unit vector direction which maximises the OV circuit $W_O W_V$ to output a vector in the direction of h_A .

Definition (Read direction). Let $W_O W_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ be the OV circuit of head h . The *read direction* $r_A^h \in \mathbb{R}^{d_{\text{model}}}$ for task A is the input direction to $W_O W_V$ whose image is maximally aligned with the head vector, subject to having no projection along the kernel of $W_O W_V$:

$$r_A^h = \arg \max_{\substack{z \in \mathbb{R}^{d_{\text{model}}}, \|z\|=1 \\ z \perp \ker(W_O W_V)}} \cos(W_O W_V z, h_A). \quad (1)$$

The kernel constraint is there to make the argmax unique and well defined. The OV circuit is low rank since W_O and W_V are rectangular, so $W_O W_V$ has rank at most d_{head} and its kernel has dimension at least $d_{\text{model}} - d_{\text{head}}$ ($4096 - 256 = 3840$ for GPT-J). Without the constraint adding any kernel vector u to a maximiser z^* leaves the output unchanged ($W_O W_V(z^* + u) = W_O W_V z^*$), but I would not say that u is part of the task identification subspace since it has no causal downstream impact if it gets ignored by M .

The maximum cosine of 1 is attained by the normalised truncated pseudo inverse (where you do SVD on $W_O W_V$ and remove all singular values of 0 and then take the inverse).

$$r_A^h = \frac{(W_O W_V)^+ h_A}{\|(W_O W_V)^+ h_A\|}. \quad (2)$$

In practice, $(W_O W_V)^+$ scales components by $1/\sigma_i$, so near-zero singular values get amplified. (I believe this is the point that Dan was making earlier but I misunderstood due to the below)

Previous misunderstanding

- 1) Previously, I then took this definition and conflated maximising cosine similarity with the dot product. This gives the solution $r_A^h = (W_O W_V)^\top h_A$ (unit normalised), since that is the unit vector maximising the dot product $\langle W_O W_V z, h_A \rangle = \langle z, (W_O W_V)^\top h_A \rangle$. I assumed the unit-norm constraint on z made the two equivalent. It does not (the constraint fixes the norm of the input z , but the cosine normalises by the norm of the output $W_O W_V z$, which still varies with z). The dot product rewards output magnitude while the cosine is indifferent to it, so the two problems have different solutions, and the transpose formula solves only the dot-product one.
- 2) I thought that if the task imitation space is m -dimensional, e.g. spanned by $\{v_j\}_{j=1:m}$, then the task identification space can be spanned by at most m vectors $\{(W_O W_V)^+ v_j\}_{j=1:m}$. But this is only inverting the task subspace for a single head when in reality there are multiple heads, so you can invert a single direction in the task imitation space v_j as $\{(W_O^h W_V^h)^+ v_j\}_{h \in H}$ which means that you can have a larger dimension task identification subspace.

3. Per Prompt Read Direction

One of the issues mentioned in the previous section is that there are multiple heads involved in the computation of one function vector. After some thought, I had the following idea for *per prompt read directions*, which are basically just the

read direction that maximises the cosine similarity to the per prompt function vectors when passed through all of the heads that are selected. To formulate this mathematically, let

$$M = \sum_{h \in H} W_O^h W_V^h \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}} \quad (3)$$

be the summed OV circuit of the selected FV heads H . (Note that summing across heads treats every head as reading the *same* residual-stream vector z , even though the heads sit at different layers)

Definition (Per-prompt read direction). The *per-prompt read direction* $r_A^j \in \mathbb{R}^{d_{\text{model}}}$ for prompt p_A^j is the unit input direction to M whose image is maximally aligned with the per-prompt function vector v_A^j , subject to having no projection along the kernel of M :

$$r_A^j = \arg \max_{\substack{z \in \mathbb{R}^{d_{\text{model}}}, \|z\|=1 \\ z \perp \ker(M)}} \cos(Mz, v_A^j). \quad (4)$$

As in the single-head case, the maximum is attained by the normalised truncated pseudo inverse:

$$r_A^j = \frac{M^+ v_A^j}{\|M^+ v_A^j\|}. \quad (5)$$

Replacing v_A^j with v_A gives the corresponding task-level read direction r_A under M .

Again, using the no projection in the kernel space ensures uniqueness, so that each prompt has one read direction.

Caveat (near-kernel). The kernel constraint resolves multiplicity from *exact* zeros only. A full-rank M can still have very near zero singular values: M^+ scales by $1/\sigma_i$, so the exact solution can be dominated by directions the circuit barely transmits. There might be some optimisation do to around this: e.g. a robust variant could be the τ -truncated pseudo inverse M_τ^+ (invert only $\sigma_i \geq \tau$; equivalently $z \perp$ the right-singular subspace with $\sigma_i < \tau$).